# DECA Lab Logbook — Spring Term

*Lab 3 | Task 5: Multiple Interrupt Sources (int-test2)*

## Task 5: Multiple Interrupt Sources (int-test2)

### 5.1 Program Overview

The eep1x sheet of lab3i is loaded with int-test2.ram. This design handles both IRQ0 and IRQ1. The single ISR at 0xFF00 uses a conditional branch to determine which interrupt source is active and dispatches to ISR0 or ISR1 accordingly. Interrupts are not nested in this design.

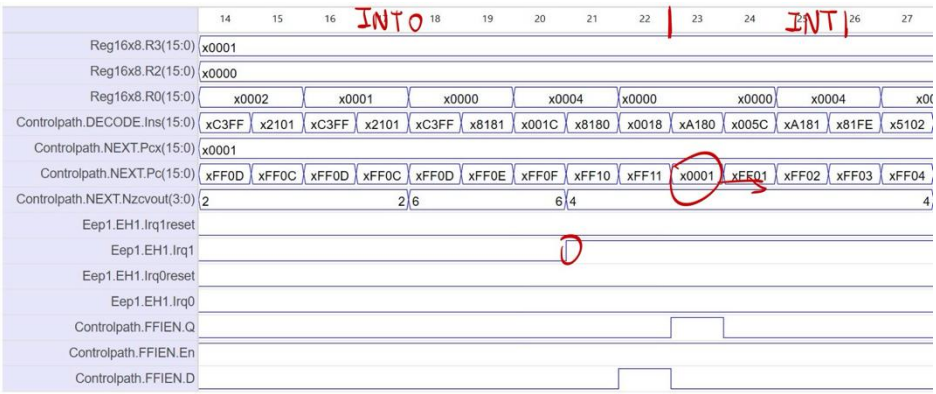
```
Int-test2:
// test interrupt 0 & 1
         // interrupts are not nested.
         // isr has conditional jump to determine which interrupt
         // is taken.
         // Base program
         SETI // set FFIEN.Q (enable interrupts)
    LOOP ADD R0, #1 // increment R0
         JNE LOOP

         ORG 0xff00 // code from here goes into address 0xff00
    ISR  STR R0, [0xFF80] // save base program R0
         MOV R0, Flags // save base program flags
         STR R0, [0xFF81]
         LDR R0, [0xFFFE]
         AND R0, #2 // is IRQ1 active?
         JNE ISR1 // if so jump to ISR1
         // otherwise, ISR0 must be active.
    ISR0 MOV R0, #1
         STR R0, [0xFFFF] // Interrupt 0 reset high
         MOV R0, #0
         STR R0, [0xFFFF] // Interrupt 0 reset low
         // Payload for interrupt 0
         // increment mem[0xFF82] by 1
         ADD R3, #1
         // small loop to add some time to INT0
         MOV R0, #3
         SUB R0, #1
         JNE -1
         // end of payload
    INTEND0
         LDR R0, [0xFF81]
         MOV Flags, R0 // restore base program flags
         LDR R0, [0xFF80] // restore R0
         RETINT
    ISR1
         MOV R0, #2
         STR R0, [0xFFFF] // Interrupt 1 reset high
         MOV R0, #0
         STR R0, [0xFFFF] // Interrupt 1 reset low
         // Payload for interrupt 1
         ADD R4, #1
         // end of payload
```

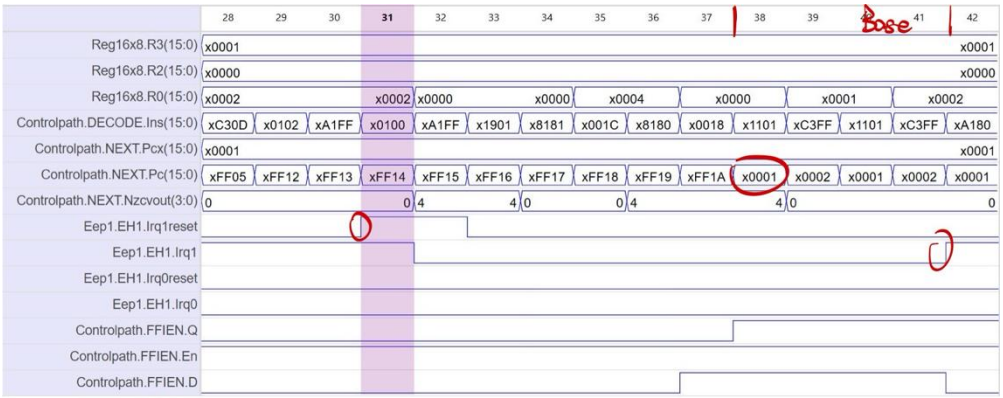
```
INTEND1
     LDR R0, [0xFF81]
     MOV Flags, R0 // restore base program flags
     LDR R0, [0xFF80] // restore base program R0
     RETINT // return to base program and set FFIE
```

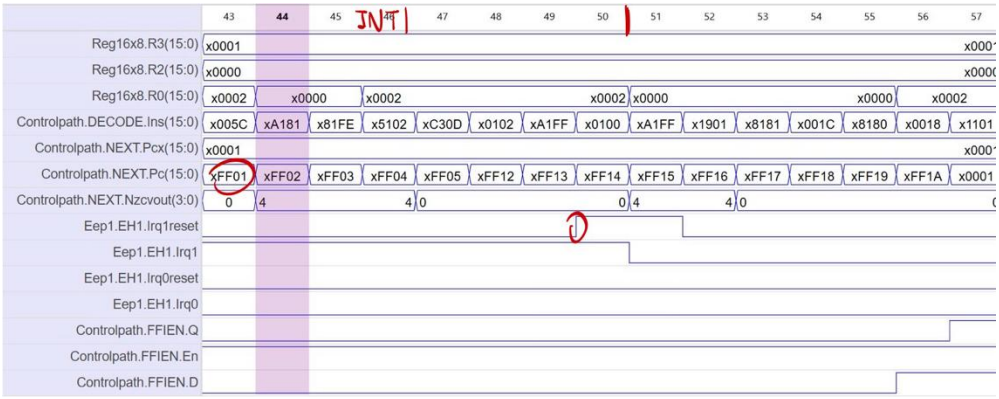
## 5.2 Cycle-by-Cycle Execution Analysis (First 50 Cycles)

***Simulate the eep1x (not eep1) sheet of lab3i running int-test2.ram in the waveform simulator. View FFIEN, IRQ0, IRQ0reset, IRQ1, IRQ1reset, NZCVout, and PCX in addition to PC, INS, R0, R2, and R3. From the first 50 clock cycles determine which cycles execute the base program, which execute INT0 servicing, and which execute INT1 servicing. Confirm that after each interrupt return, R0 and the N,Z,C,V flags remain unchanged.***

| Cycle Range | Execution Phase | Evidence |
|---|---|---|
| 0 | Base program | PC = 0x0000–0x0001; FFIEN goes high after SETI |
| 1 – 22 | INT0 servicing (ISR0) | PC jumps to 0xFF01; IRQ0reset pulse observed; R3 increments by 1 (ADD R3, #1 in ISR0) |
| 23 – 37 | INT1 servicing (ISR1) | IRQ1reset pulse observed; R4 increments by 1 (ADD R4, #1 in ISR1) |
| 38 – 41 | Base program | PC returns to base program address; R0 and flags restored |
| 42 – 50 | INT1 servicing (ISR1) | PC jumps back to 0xFF01; IRQ1reset pulse; R4 increments again |






## 5.3 Confirmation: R0 and NZCV Unchanged After Each Return

**After INT0 RETINT (cycle 23):** R0 returns to its pre-interrupt base-program value. NZCVout returns to the same value as before the interrupt, confirmed by the waveform.

**After INT1 RETINT (cycle 37 and cycle 50):** The same restoration is confirmed. R0 resumes its base-program value and NZCV flags are unchanged.

This confirms that the context save-and-restore mechanism (STR/LDR for R0; MOV R0,Flags and MOV Flags,R0 for NZCV) correctly preserves all base program state across both INT0 and INT1 service routines.